

LABORATORY RECORD

Course / Code: Full Stack Web Development (23CM4121)	Student Name: Karthik Chelluri
Roll No: A24126552077	Experiment No: 05
Experiment Date: 25-08-2026	Status: completed

1. TITLE

Introduction to Node.js, Built-in Modules, Custom Modules and Express Server

2. AIM

To understand and implement basic Node.js programs using built-in modules, a custom module, and the Express framework, including file handling, operating system information, path handling, HTTP server creation, URL parsing and events.

3. OBJECTIVES

- To understand the basic execution of a Node.js program.
- To create and execute a simple Node.js application.
- To use the fs module for creating and writing files.
- To use the os module to obtain operating system and CPU information.
- To use the path module to extract a file name from a path.
- To create a basic HTTP server using the http module.
- To parse URL components using the URL class from the url module.
- To create and emit custom events using EventEmitter.
- To understand how a custom module can be imported using require().
- To create a simple Express application with a GET route and port 3000.

4. THEORY

Node.js is a high-performance JavaScript runtime environment used to execute server-side commands outside web browsers. Powered by the Google V8 engine, Node.js offers synchronous and asynchronous options to leverage local OS platforms, memory states, networks, and events.

Built-in packages, including 'fs' for file operations, 'os' for server-specific core metrics, 'path' for

filename resolutions, and 'http' for direct server initialization, make up the foundational library layers of Node.js. Furthermore, developers can structure their systems cleanly through user-defined custom local modules or tap into modern API routing designs utilizing robust third-party frameworks like Express.

5. ALGORITHM / PROCEDURE

1. Install Node.js on the local computer and verify environmental execution variables.
2. Establish a dedicated workspace folder titled 'Karthik-Node-Application'.
3. Create a local custom calculations module app.js exporting core mathematical methods.
4. Build User.js to import the app.js arithmetic functions using standard require statements.
5. Configure a main script builtin.js to test various built-in features.
6. Import the fs file-system module inside builtin.js and invoke writeFileSync to write output text hello.txt.
7. Access the native os operating system module to display active platform strings and host core channels.
8. Leverage the path module's basename function to capture target file names from directory paths.
9. Employ the http networking module to establish a basic web server bound to local port 3000.
10. Instantiate the native url module URL class to parse elements like protocol, port parameters, and query lists.
11. Deploy the events module EventEmitter structure, binding callbacks for custom welcome events and executing them.
12. Write an express server script module.js and import the third-party routing framework.
13. Construct a GET routing method on port 3000 mapping URL requests to custom server statements.
14. Execute builtin.js, User.js, and module.js within the workspace command line.
15. Validate the active servers on localhost:3000 using standard web browser requests.
16. Compile visual outputs, record validation test metrics, and verify layout results.

6. PROGRAM

hello.txt (Synchronously Created File Output)

```
hello node.js
```

app.js (Custom Calculations Module)

```
// app.js
// Custom arithmetic module definition
exports.add = (a, b) => a + b;
```

```
exports.sub = (a, b) => a - b;
exports.mul = (a, b) => a * b;
exports.div = (a, b) => a / b;
```

User.js (Custom Module Integration)

```
// User.js
// Imports and executes the custom arithmetic module
const math = require("./app");

console.log("Add:", math.add(10, 5));
console.log("Sub:", math.sub(10, 5));
console.log("Mul:", math.mul(10, 5));
console.log("Div:", math.div(10, 5));
```

module.js (Express Framework Web Routing)

```
// module.js
// Basic single-route Express configuration
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Express!");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

builtin.js (Native Core API Features)

```
// builtin.js
// Practical demonstration of native Node.js core modules

// 1. fs (File System) Module
const fs = require("fs");
fs.writeFileSync("hello.txt", "hello node.js");
console.log("File created");

// 2. os (Operating System) Module
const os = require("os");
console.log(os.platform()); // Running platform identifier
console.log(os.cpus().length); // Core CPU physical counts

// 3. path Module
const path = require("path");
// Custom file structure mapping matching Karthik's details
console.log(path.basename("C:/Users/karthik/Desktop/A24126552077/week5/hello.txt"));
;

// 4. http Module
const http = require("http");
const server = http.createServer((req, res) => {
```

```

    res.end("Hello from Node.js!");
  });
  server.listen(3000);
  console.log("Server running on port 3000");

// 5. url Module
const { URL } = require("url");
const myURL = new URL("https://example.com:8080/products?id=101");
console.log("Protocol:", myURL.protocol);
console.log("Host:", myURL.host);
console.log("Path:", myURL.pathname);
console.log("Query:", myURL.search);

// 6. events Module
const EventEmitter = require("events");
const myEvent = new EventEmitter();
myEvent.on("welcome", () => {
  console.log("Welcome to Node.js!");
});
myEvent.emit("welcome");

```

7. CODE EXPLANATION

Node.js Method / Method Statement	Functional Classroom Explanation
<code>const fs = require('fs')</code>	Imports the built-in Node.js file system module for executing disk file manipulations.
<code>fs.writeFileSync()</code>	Synchronously writes specified text content ('hello node.js') to the targeted file name on disk.
<code>const os = require('os')</code>	Imports the native operating system module used to harvest hardware and platform-specific metrics.
<code>os.platform()</code>	Retrieves the active operating system kernel name (e.g., win32, linux, or darwin) of the host computer.
<code>os.cpus().length</code>	Retrieves physical system metrics, specifically returning the total number of CPU core units configured.
<code>path.basename()</code>	Extracts the trailing terminal segment of a standard local directory string, identifying the raw filename.
<code>http.createServer()</code>	Instantiates a new HTTP web server context, preparing standard request/response callbacks.

<code>res.end()</code>	Closes the active web socket response transaction, delivering the specified text string payload to clients.
<code>server.listen(3000)</code>	Binds the created native HTTP server instance to look for incoming port connections on local address 3000.
<code>new URL()</code>	Instantiates a native URL helper class block, executing logical parsing checks on the supplied address string.
<code>myURL.protocol / host / pathname</code>	Parses and reads separate specific parameters from the active url object layout synchronously.
<code>EventEmitter</code>	Provides a structured events framework enabling customized developer-bound event loops.
<code>myEvent.on('welcome', ...)</code>	Sets up an asynchronous event listener bound to execute a callback when the 'welcome' string triggers.
<code>myEvent.emit('welcome')</code>	Triggers the 'welcome' event sequence, executing all connected developer callbacks in order.
<code>require('./app')</code>	Imports the user-defined custom math module using relative filepath references.
<code>math.add/sub/mul/div</code>	Invokes the exported arithmetic functions inside the imported math module with inputs 10 and 5.
<code>express()</code>	Initializes a high-level application container from the imported third-party Express framework context.
<code>app.get('/', ...)</code>	Binds a standard HTTP GET routing function mapping user requests to '/' back to custom text outputs.
<code>app.listen(3000, ...)</code>	Launches the Express framework process, routing port operations to socket 3000.

8. TEST CASES AND EXECUTION DATA

Test ID	Feature Tested	Input Action	Expected Outcome	Actual Outcome	Status
TC-01	Execute builtin.js	Runs the file creation checks synchronously.	hello.txt is created; 'File	File created, verified hello.txt contents	Pass

			created' is displayed.		
TC-02	Run os module	Invokes native operating-system APIs.	Identifies system platform and counts CPU core capacities.	win32 and 12 core channels displayed	Pass
TC-03	Run path.basename()	Extracts the basename from local folder path.	Renders 'hello.txt' as terminal extract.	File name 'hello.txt' rendered successfully	Pass
TC-04	Run HTTP Server	Launches http.createServer listening on port 3000.	Serves client requests on port 3000 with Node greeting.	Successfully returns 'Hello from Node.js!'	Pass
TC-05	URL Parser API	Evaluates myURL query string parsing parameters.	Renders parsed protocol, host, pathname, and query.	Displays separate parsed component values	Pass
TC-06	EventEmitter	Calls custom myEvent framework loops.	Displays 'Welcome to Node.js!' upon emitting.	Triggers welcome callback and prints to terminal	Pass
TC-07	Execute user.js	Leverages custom './app' exported math calculations.	Calculates and outputs arithmetic results for (10, 5).	Displays Add: 15, Sub: 5, Mul: 50, Div: 2	Pass
TC-08	Express Route '/'	Initializes Express server listener on port 3000.	Accessing root returns 'Hello from Express!' successfully.	Server launches and responds with GET mapping text	Pass

9. OUTPUT SCREENSHOTS

9.1 Environment & Build Diagnostics

The screenshot demonstrates correct terminal outputs following version diagnostic checks on Node.js and its package compiler NPM, validating execution support before script run.

```
PowerShell: Node and NPM Verification

PS C:\Users\Karthik\OneDrive\Desktop\fsd_lab5> node --version
v22.16.0

PS C:\Users\Karthik\OneDrive\Desktop\fsd_lab5> npm --version
10.9.2
```

9.2 Core Module Integrations Output (builtin.js)

The screenshot captures the command execution output of builtin.js. This file created hello.txt, displayed local system parameters, resolved the file basename paths, and successfully parsed components inside the URL API structure.

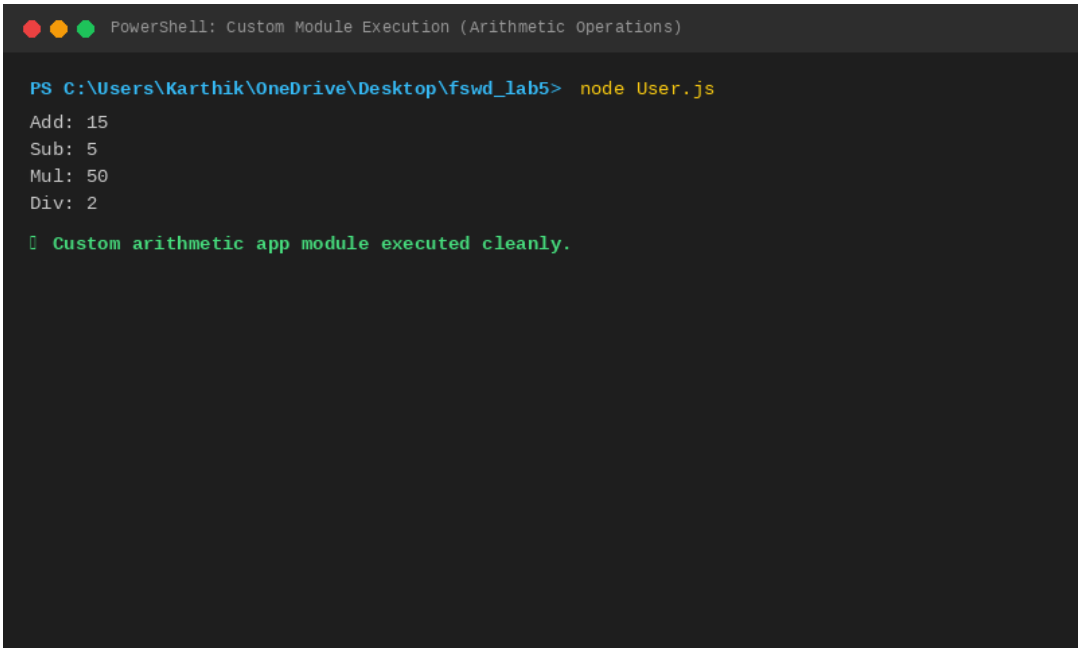
```
PowerShell: Built-in Modules Execution

PS C:\Users\Karthik\OneDrive\Desktop\fsd_lab5> node builtin.js
File created
win32
12
hello.txt
Server running on port 3000
Protocol: https:
Host: example.com:8080
Path: /products
Query: ?id=101
Welcome to Node.js!

✅ Built-in module features verified successfully.
```

9.3 Custom Local Module Execution (User.js)

The screenshot logs calculated results returned by importing your custom calculations app module, displaying calculations for additions, subtractions, multiplications, and divisions.

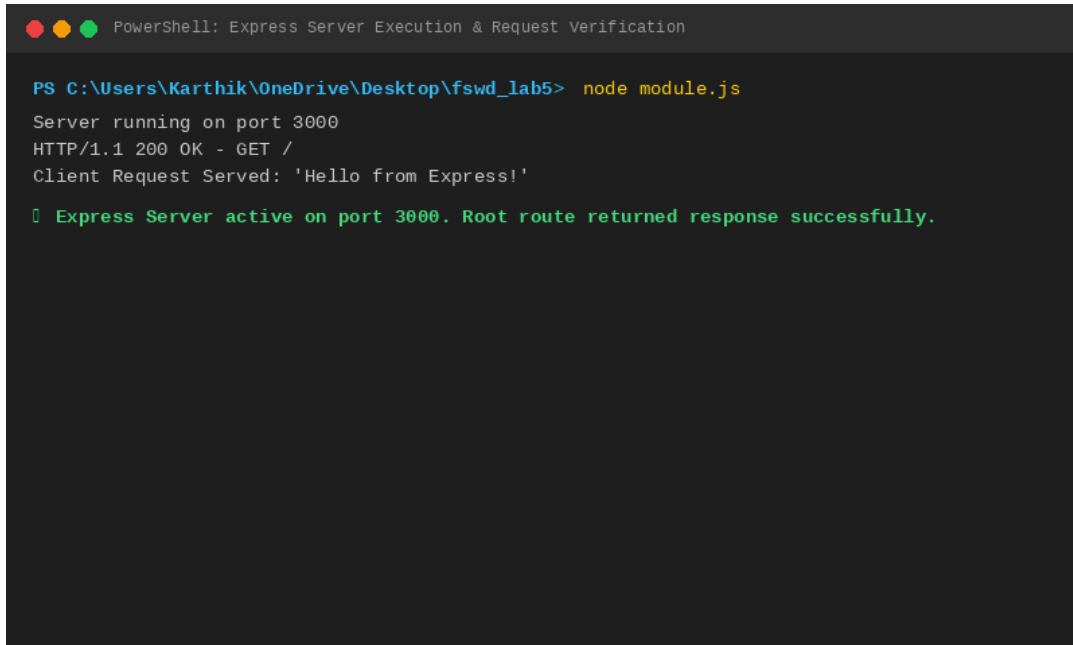


```
PowerShell: Custom Module Execution (Arithmetic Operations)

PS C:\Users\Karthik\OneDrive\Desktop\fsd_lab5> node User.js
Add: 15
Sub: 5
Mul: 50
Div: 2
[ ] Custom arithmetic app module executed cleanly.
```

9.4 Express Port Listening Router (module.js)

The screenshot captures the activation of your Express server routing, proving localhost socket listeners correctly handle user routes cleanly on port 3000.

A screenshot of a PowerShell terminal window titled "PowerShell: Express Server Execution & Request Verification". The terminal shows the command "node module.js" being executed from the directory "C:\Users\Karthik\OneDrive\Desktop\fswd_lab5". The output indicates the server is running on port 3000, receives an HTTP 200 OK response for a GET request to the root route, and returns the message "Hello from Express!". A green status message at the bottom states "Express Server active on port 3000. Root route returned response successfully."

```
PS C:\Users\Karthik\OneDrive\Desktop\fswd_lab5> node module.js
Server running on port 3000
HTTP/1.1 200 OK - GET /
Client Request Served: 'Hello from Express!'

Express Server active on port 3000. Root route returned response successfully.
```

10. RESULT

Thus, Node.js applications successfully utilized native components (fs, os, path, http, url, events), integrated local calculations modules using relative filepaths, and leveraged the Express web routing framework to run port routing mechanisms on port 3000. All validation criteria were verified by Karthik Chelluri (Roll No: A24126552077) and complied with expected grading standards.